

Design a Chat System - Step 2: Propose High-Level Design and Get Buy-In

1. Basic Communication Model

In a chat system:

- Clients can be:
 - Mobile applications
 - Web applications
 - Desktop applications

Important Point

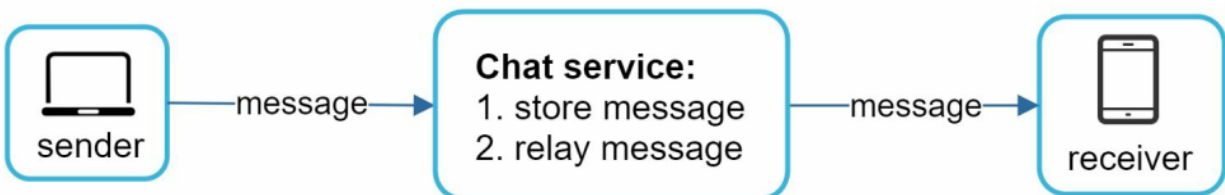
Clients do **NOT** communicate directly with each other.

Instead:

None

Client ↔ Chat Service ↔ Client

The chat service acts as the central communication system.



2. Responsibilities of Chat Service

The chat service must support the following core functionalities:

Receive Messages

Receive messages sent by users.

Find Correct Recipients

Identify intended receivers for every message.

Deliver Messages

Relay messages to recipient devices.

Store Offline Messages

If a user is offline:

- Messages are temporarily stored on the server
- Delivered once the user reconnects

3. Sender Side Communication

When a sender sends a message:

None

Sender Client → Chat Service → Receiver Client

Protocol Used

The sender side commonly uses:

None

HTTP Protocol

Why HTTP?

HTTP is:

- Reliable
- Simple
- Widely supported
- Easy to scale

Keep-Alive Connection

The system uses:

None

HTTP Keep-Alive

This allows:

- Persistent connections
- Reduced TCP handshakes
- Better performance

Example Flow

1. Client opens HTTP connection
2. Sends message to chat server
3. Chat server processes and routes message

Real-World Usage

Many popular chat systems initially used HTTP for sending messages.

Example:

- Facebook Messenger

4. Receiver Side Problem

The receiver side is more difficult.

Problem with HTTP

HTTP is:

None

Client-Initiated

Meaning:

- Server cannot directly push data to client easily

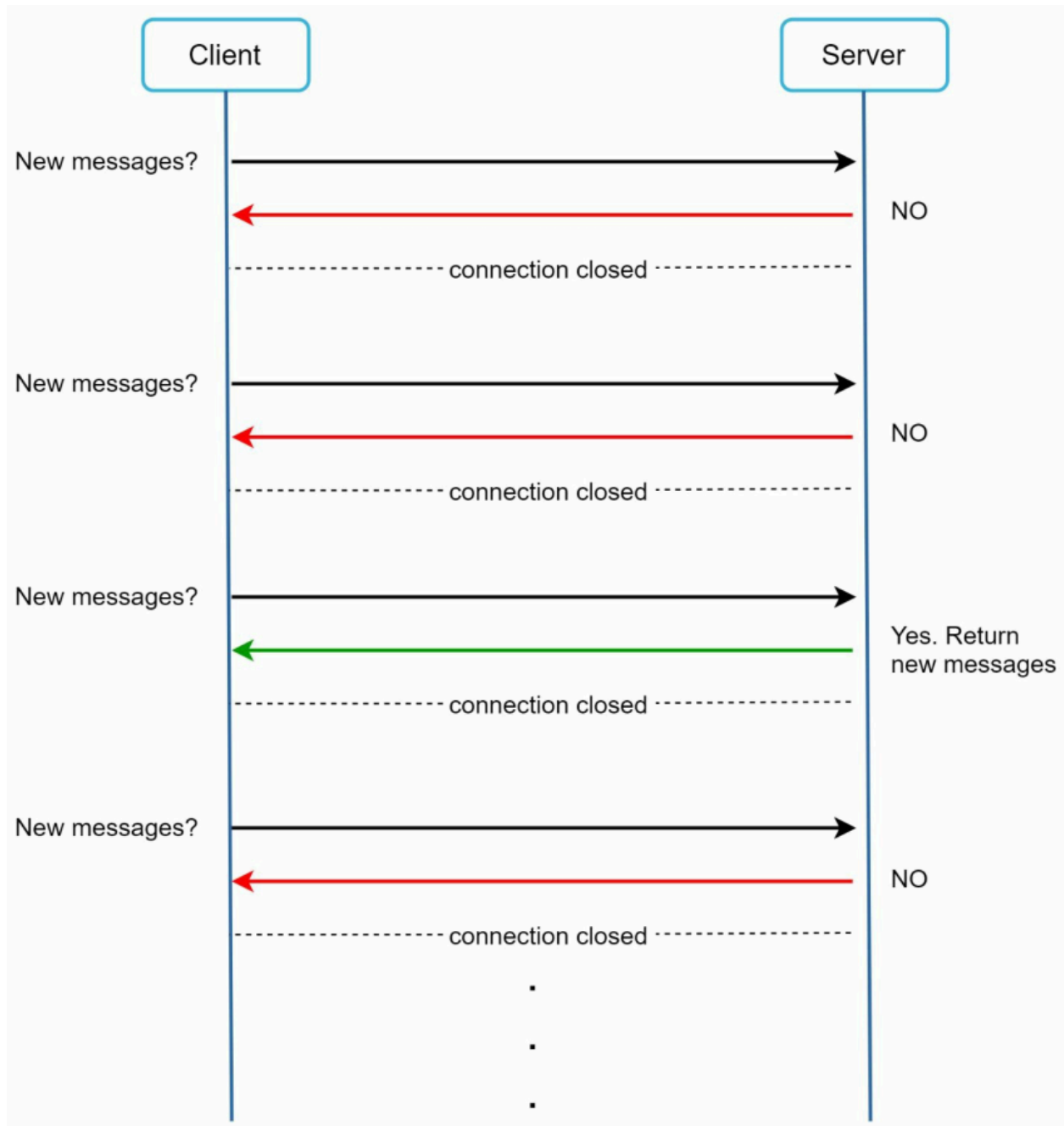
Challenge

How can server instantly notify clients about new messages?

To solve this, several techniques are used:

1. Polling
2. Long Polling
3. WebSocket

5. Polling



How Polling Works

The client repeatedly asks:

None

"Do I have new messages?"

Example:

None

Every 2 seconds → request server

Polling Flow

1. Client sends request
2. Server checks for messages
3. Server returns response
4. Client repeats process continuously

6. Problems with Polling

High Resource Usage

Most requests return:

None

"No new messages"

Yet server resources are consumed.

Inefficient

Frequent requests increase:

- CPU usage
- Network traffic
- Server load

Scalability Issues

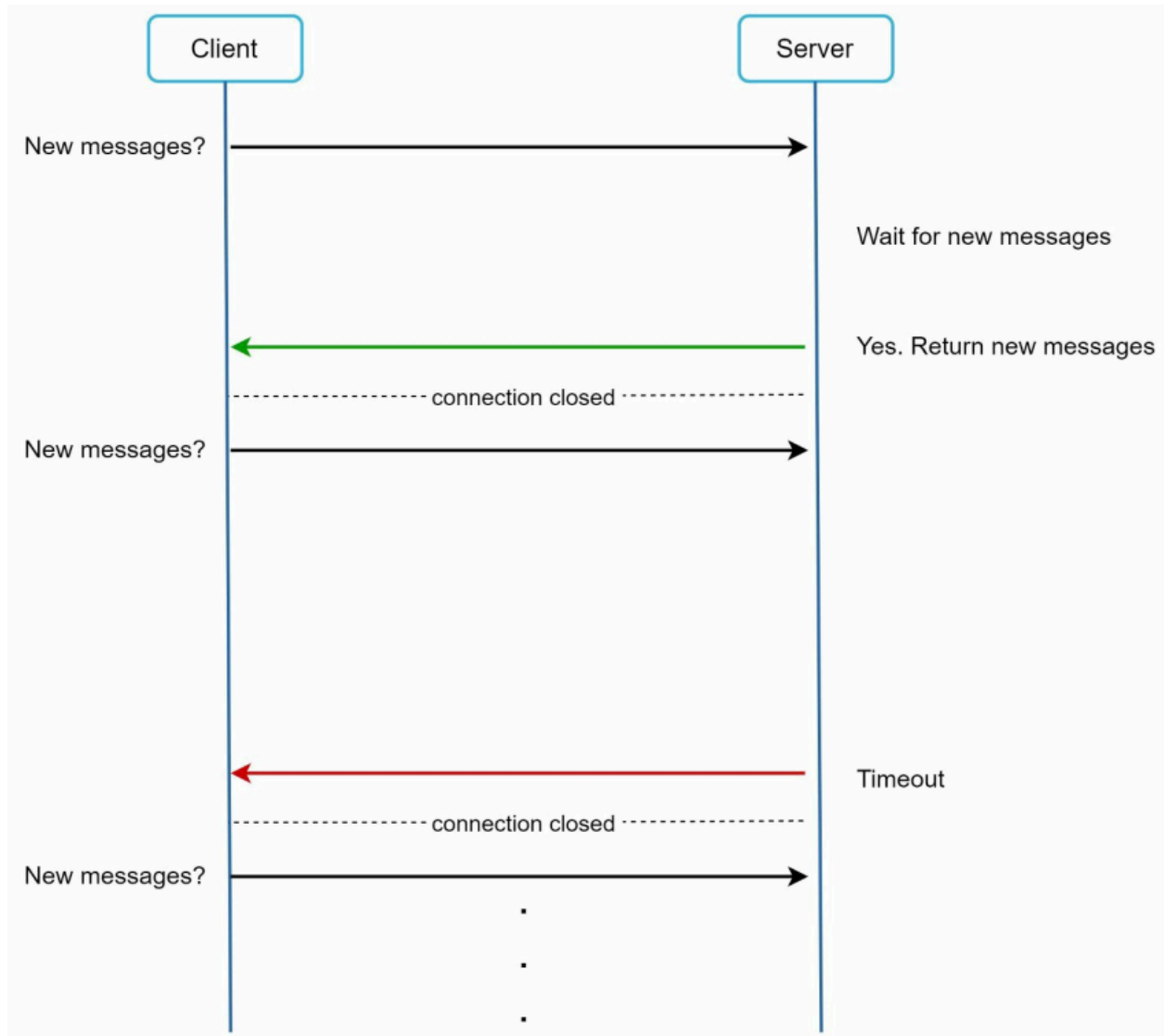
At massive scale:

None

Millions of users × repeated requests

becomes extremely expensive.

7. Long Polling



Long polling improves normal polling.

8. How Long Polling Works

Instead of immediately responding:

- Server keeps connection open
- Waits until:
 - New message arrives
 - Timeout occurs

Long Polling Flow

1. Client sends request
2. Server waits
3. If new message arrives:
 - Server responds immediately
4. Client instantly opens another connection

9. Advantages of Long Polling

Fewer Empty Responses

Connections remain open until data exists.

Better than Regular Polling

Reduces unnecessary repeated requests.

10. Problems with Long Polling

Despite improvements, long polling still has limitations.

Stateless Server Problem

HTTP servers are usually stateless.

Using load balancing:

None

Sender → Server A

Receiver Long Poll → Server B

The receiving server may not hold the active connection.

Difficult to Detect Disconnects

Server cannot reliably determine:

- Whether client disconnected
- Whether network failed

Still Inefficient

Inactive users continue reconnecting after timeout periods.

This wastes:

- Network bandwidth
- Server resources

11. Comparison: Polling vs Long Polling

Feature	Polling	Long Polling
Server Efficiency	Low	Better
Real-time Capability	Weak	Moderate

Resource Usage	High	Medium
Complexity	Simple	Moderate
Scalability	Poor	Better but limited

12. Key Design Insight

Traditional HTTP approaches are not ideal for real-time chat systems because:

- HTTP is request-response based
- Server push is difficult
- Persistent bidirectional communication is needed

This leads to the need for:

None

WebSocket

which enables full-duplex real-time communication.

Key Takeaway

Chat systems require:

- Persistent connections
- Low latency
- Efficient message delivery

Polling and long polling improve communication incrementally, but both have scalability and efficiency limitations.

Modern chat systems prefer:

None

WebSocket-based communication

for true real-time messaging.